```
        concat(
          post { ...
          },
          get { ...
          })
      }

    val bindingFuture: Future[ServerBinding] =
      Http().newServerAt("localhost", 8080).bind(route)                ❺

    println(s"server at localhost:8080 \nPress RETURN to stop")
    StdIn.readLine()
    bindingFuture
      .flatMap(_.unbind())
      .onComplete(_ => system.terminate())
  }
}
```

❶ To run the future when binding the route

❷ To run the .ask in the get

❸ Deals with serialization

❹ Definition of routes

❺ Starts the server and sets the route

Now that you know how to create an Akka HTTP port, it's time to look at the other types of ports you can create.

11.2 Akka gRPC

Akka gRPC is another option that allows you to create ports to communicate with your `ActorSystem`. As with Akka HTTP, you can create both the server and the clients. In this section, you learn about the server side. The client side is discussed in chapter 16.

The advantages of this module are numerous. You can use HTTP/2, and it creates all the necessary classes once you have defined the interface you want. This is done in a so-called *.proto file*. Let's start